

auk Full Workflow: AOS 2026 workshop

Aimee M. Van Tatenhove & Fabiola Rodríguez Vásquez

Table of contents

Datasets and resources	1
Data	2
Scripts	2
Section 1. Import and explore eBird raw data	2
1.1. Load EBD (eBird Basic Dataset) and sampling files	3
1.2. Exploring the structure of eBird data	4
1.3. Pipes- the building block of the ‘auk’ workflow	10
Section 2. Setup and execute filters for eBird raw data in ‘auk’	11
2.1. Set path	12
2.2. Define ‘auk’ filters	12
2.3. Apply filters and save new dataset	15
Section 3. Zerofilling eBird raw data to obtain a “presence-absence” dataset	16
3.1. Set the path for the files	17
3.2. Applying filters	17
3.3. Zerofill to generate the presence-absence dataset	20
3.4. Final visualization	26

This is the full workflow for the AOS 2026 workshop, “Setting Up Your Analyses Up for Success with ‘auk’”. The workshop is divided into four sections to:

1. Learn how to import ‘auk’ data and explore its file structure.
2. Learn how to use ‘auk’ filters to sample the eBird data for a project.
3. Learn how to process the eBird raw data (**ebd**) and checklists events (**sampling**) files to obtain a dataset for species “presences-absences”.

Datasets and resources

Participants will find all relevant resources in the provided folder that includes an R project, data and code.

Data

The process to obtain raw data from eBird's website can take multiple days, so the instructors downloaded and processed eBird datasets for this workshop. There are two datasets:

- **ebd** and **sampling** files for three species found in the eastern United States.
 - ebd_filtered_us_AOS_2026.txt
 - effort_filtered_us_AOS_2026.txt
- **ebd** and **sampling** files for two species found in the seven Central American countries.
 - ebd_filtered_cam_AOS_2026.txt
 - effort_filtered_cam_AOS_2026.txt

Additional to the eBird data, geospatial files will be used for the applications sections of this workshop and can be found in the folder “gis”.

Scripts

Separate Quarto documents (.qmd) have been prepared for each of the objectives mentioned above. These documents have brief explanations or text that introduces an exercise and are followed by ‘code chunks’, spaces where R will be executed. Feel free to modify text below or above the code chunks to add your own notes, or add comments (using #) to your code within the code chunks.

Section 1. Import and explore eBird raw data

The first step in the ‘auk’ workflow is to call the necessary libraries for the R packages. If you followed the check_auk-runs.R script sent before the workshop and followed the steps during the “preparing the environment” section of the workshop, all necessary packages should be ready. If not call the function install.packages() for the necessary libraries below.

```
library(tidyverse)
library(auk)
source("custom-functions.R")

set_theme(custom.theme())
```

1.1. Load EBD (eBird Basic Dataset) and sampling files

‘auk’ requires direction to where the downloaded data is. There are two files we will work with by the end of this workshop, one will be called ebd which is a shorthand for eBird Basic Dataset and has the species observation data, the other will be called sampling or effort and has the data for the sampling information compiled with each eBird checklist. In other tutorials this file can also be called sed for sampling event data, but in this workshop we will use sampling or effort interchangeably as these two terms are important for designing filters and future data analyses.

Create paths for the ebd and sampling files provided by the instructors. When you work with your own data these two files will have names that reference the location and year of your request.

```
## Set paths for EBD & effort data
in_ebd_small <- "data/ebd_filtered_cam_AOS_2026.txt"
in_effort_small <- "data/effort_filtered_cam_AOS_2026.txt"
```

The data has NOT been loaded yet! Remember that ‘auk’ is meant to be an effective way of processing eBird large datasets, but as we can see below these first sample datasets are not too big.

```
## Smallish files, so we can load into R memory directly
get.filesize(in_ebd_small)
```

```
[1] "17.44 MB"
```

```
get.filesize(in_effort_small)
```

```
[1] "111.57 MB"
```

To read these files we will use `read_abd()` and `read_sampling()`.

Take a look at the help file for `read_ebd()`. It manages two things that are important to know: 1) The argument `unique=TRUE`, removes duplicated grouped checklists. 2) `rollup=TRUE`, deals with a taxonomic “rollup” so you are only working with species-level data.

```
## How do we read the files?
?read_ebd()
```

Once the files are read in, we are ready to explore them.

```
## Apply the read_ebd() function
ebd <- read_ebd(in_ebd_small)
effort <- read_sampling(in_effort_small)
```

1.2. Exploring the structure of eBird data

Take a look at all attributes (i.e. columns) that these files have.

Get a sense for the information that is collected by eBird in each checklist.

Compare the ebd and effort files.

```
glimpse(ebd)
```

```
Rows: 9,251
Columns: 52
$ checklist_id          <chr> "\"\"", "\"\"", "G13682338", "G13682606", "G~
$ global_unique_identifier <chr> "URN:CornellLabOfOrnithology:EBIRD:OBS274390~
$ last_edited_date      <chr> "2025-10-23T13:29:39.261520Z", "2025-08-
10T1~
$ taxonomic_order       <dbl> 10836, 884, 884, 10836, 10836, 10836, 884, 8~
$ category              <chr> "species", "species", "species", "species", ~
$ taxon_concept_id      <chr> "avibase-20DB2669", "avibase-01A38979", "avi~
$ common_name           <chr> "Northern Emerald-Toucanet", "Crested Guan",~
$ scientific_name        <chr> "Aulacorhynchus prasinus", "Penelope purpura~
$ exotic_code           <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ observation_count     <chr> "1", "1", "4", "1", "2", "2", "2", "2", "3",~
$ breeding_code         <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ breeding_category     <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ behavior_code         <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ age_sex               <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ country               <chr> "Costa Rica", "Costa Rica", "Costa Rica", "N~
$ country_code          <chr> "CR", "CR", "CR", "NI", "CR", "GT", "CR", "B~
$ state                 <chr> "Puntarenas", "Alajuela", "Guanacaste", "Nue~
$ state_code            <chr> "CR-P", "CR-A", "CR-G", "NI-NS", "CR-
C", "GT~
$ county               <chr> NA, NA, NA, NA, NA, "\"\"", NA, "\"\"", NA, ~
$ county_code          <chr> NA, NA, NA, NA, NA, "\"\"", NA, "\"\"", NA, ~
$ iba_code              <chr> "BIRDLIFE_20407", "BIRDLIFE_20406", "BIRDLIF~
$ bcr_code              <int> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ usfws_code            <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ atlas_block           <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
```

```

$ locality <chr> "El Trapiche Tour", "Calle La Catarata, Upal~
$ locality_id <chr> "L5040956", "L39045713", "L6130799", "L39060~
$ locality_type <chr> "H", "P", "H", "P", "H", "H", "P", "H", "H",~
$ latitude <dbl> 10.327828, 10.727929, 10.712260, 13.777833, ~
$ longitude <dbl> -84.83845, -85.08201, -85.07161, -86.53491, ~
$ observation_date <date> 2025-01-01, 2025-01-01, 2025-01-01, 2025-
01~
$ time_observations_started <chr> "06:40:00", "06:28:00", "07:32:00", "06:54:0~
$ observer_id <chr> "obsr1180770,obsr418184,obsr4404304,obsr1499~
$ observer_orcid_id <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ sampling_event_identifier <chr> "S207433891,S207445671,S207472524,S207476364~
$ observation_type <chr> "Stationary", "Traveling", "Traveling", "Tra~
$ protocol_name <chr> "Stationary", "Traveling", "Traveling", "Tra~
$ protocol_code <chr> "P21", "P22", "P22", "P22", "P22", "P22", "P~
$ project_names <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ project_identifiers <chr> NA, NA, NA, NA, NA, "\"\"", NA, "\"\"", NA, ~
$ duration_minutes <int> 25, 30, 82, 50, 37, 235, 18, 330, 40, 248, 1~
$ effort_distance_km <dbl> NA, 0.106, 0.467, 0.930, 0.309, 2.160, NA, 5~
$ effort_area_ha <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ number_observers <int> 1, 1, 3, 2, 2, 3, 3, 5, 2, 2, 5, 2, 2, 3, 2,~
$ all_species_reported <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TR~
$ group_identifier <chr> "\"\"", "\"\"", "G13682338", "G13682606", "G~
$ has_media <lgl> FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, FAL~
$ approved <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TR~
$ reviewed <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
$ reason <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ checklist_comments <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ species_comments <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"\"~
$ v53 <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~

```

```
glimpse(effort)
```

```
Rows: 90,127
```

```
Columns: 35
```

```

$ checklist_id <chr> "\"\"", "G13679650", "G13679884", "G13680025~
$ last_edited_date <chr> "2025-01-05T17:04:49.017528Z", "2025-01-
01T0~
$ country <chr> "Costa Rica", "Costa Rica", "Costa Rica", "C~
$ country_code <chr> "CR", "CR", "CR", "CR", "CR", "CR", "GT", "C~
$ state <chr> "Cartago", "Heredia", "Cartago", "Cartago", ~
$ state_code <chr> "CR-C", "CR-H", "CR-C", "CR-C", "CR-C", "CR-
~

```

```

$ county <chr> NA, NA, NA, NA, NA, NA, "\"\"", NA, NA, NA, ~
$ county_code <chr> NA, NA, NA, NA, NA, NA, "\"\"", NA, NA, NA, ~
$ iba_code <chr> "BIRDLIFE_20419", "BIRDLIFE_20416", "BIRDLIF~
$ bcr_code <int> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ usfws_code <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ atlas_block <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ locality <chr> "Casa", "Barrio Cristo Rey", "Prusia", "Auto~
$ locality_id <chr> "L24709140", "L17427109", "L1758035", "L3905~
$ locality_type <chr> "P", "P", "H", "P", "H", "H", "H", "P", "H",~
$ latitude <dbl> 9.813258, 10.450328, 9.957325, 9.861179, 9.8~
$ longitude <dbl> -83.56494, -84.03956, -83.88541, -83.94762, ~
$ observation_date <date> 2025-01-05, 2025-01-01, 2025-01-01, 2025-
01~
$ time_observations_started <chr> "13:01:00", "00:34:00", "00:51:00", "02:32:0~
$ observer_id <chr> "obsr796872,obsr238024,obsr238024,obsr33705,~
$ observer_orcid_id <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"~
$ sampling_event_identifier <chr> "S139481685,S20491187,S20502675,S207409265,S~
$ observation_type <chr> "Stationary", "Stationary", "Traveling", "Tr~
$ protocol_name <chr> "Stationary", "Stationary", "Traveling", "Tr~
$ protocol_code <chr> "P21", "P21", "P22", "P22", "P21", "P22", "P~
$ project_names <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"~
$ project_identifiers <chr> NA, NA, NA, NA, NA, NA, "\"\"", NA, "\"\"", ~
$ duration_minutes <int> 15, 25, 39, 8, 3, 64, 7, 24, 86, 32, 104, 30~
$ effort_distance_km <dbl> NA, NA, 0.347, 0.117, NA, 0.943, NA, 0.106, ~
$ effort_area_ha <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ number_observers <int> 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ~
$ all_species_reported <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TR~
$ group_identifier <chr> "\"\"", "G13679650", "G13679884", "G13680025~
$ checklist_comments <chr> "\"\"", "\"\"", "\"\"", "\"\"", "\"\"", "\"~
$ v34 <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~

```

Notice that ebd has fewer rows than effort. This is because ebd comes from the file with species observations, effort does not have any species column, the data that was downloaded was all available checklist sampling information that matched the other criteria (location, dates) in the download request.

1.2.1 Exploring by accessing variable characteristics

Let's keep exploring the ebd file because this is the data we are sampling from for our objectives.

How many species are available in this dataset?

```
unique(ebd$common_name)
```

```
[1] "Northern Emerald-Toucanet" "Crested Guan"
```

What is the date range for this data?

```
range(ebd$observation_date)
```

```
[1] "2025-01-01" "2026-01-29"
```

How many counts does each species have?

```
table(ebd$common_name)
```

	Crested Guan	Northern Emerald-Toucanet
	6227	3024

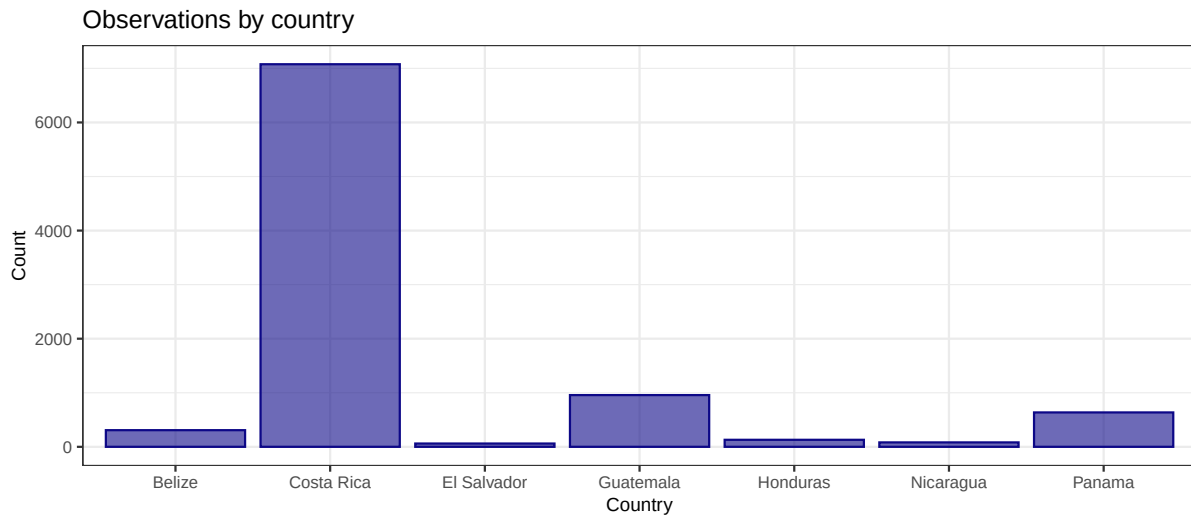
How many countries appear in this dataset?

```
#Exercise for participants
```

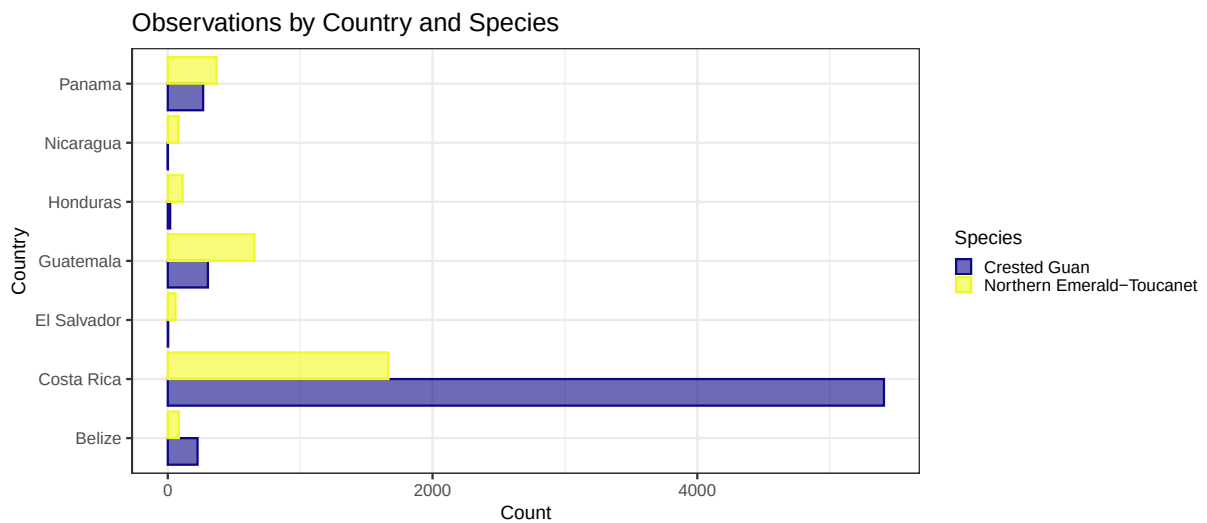
1.2.2 Exploring by plotting variables

Now for some visual exploration. Which country has the most detections?

```
## Explore visually by using ggplot's geom_bar which counts occurrences for a category
## Observations dataset
ggplot(ebd) +
  geom_bar(aes(x = country)) +
  ggtitle("Observations by country") +
  xlab("Country") + ylab("Count")
```

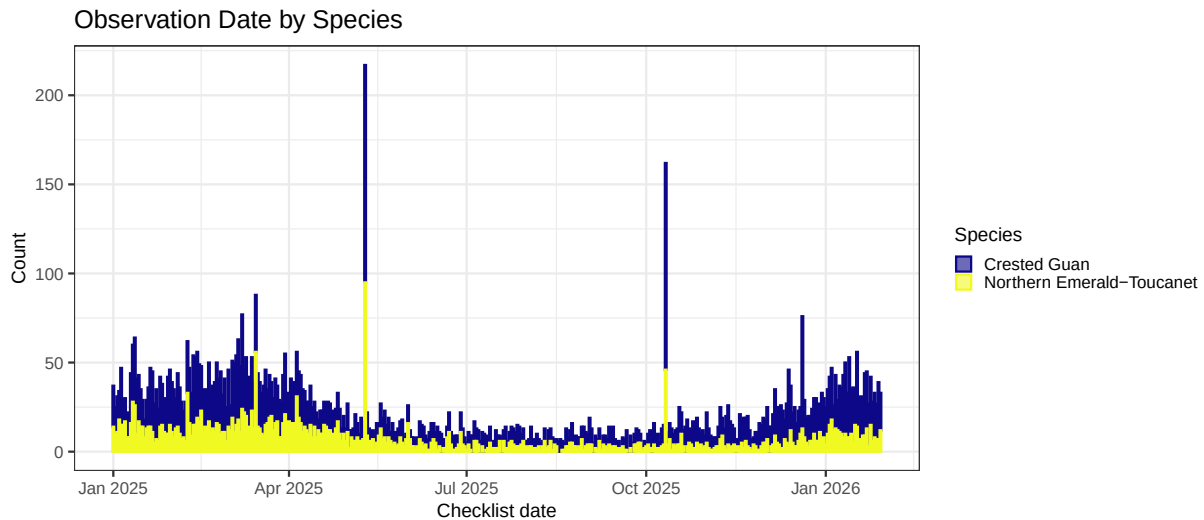


```
ggplot(ebd) +
  geom_bar(aes(x = country, color = common_name, fill = common_name),
           position = position_dodge()) +
  ggtitle("Observations by Country and Species") +
  coord_flip() +
  xlab("Country") + ylab("Count") +
  labs(color = "Species",
       fill = "Species")
```



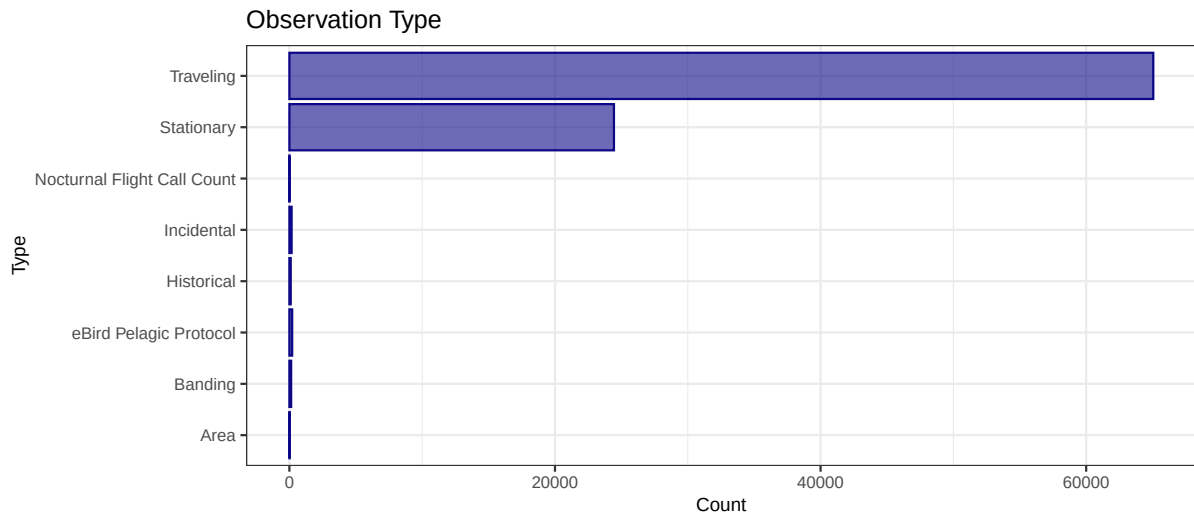
When does it seem to be a good time to observe the species?

```
ggplot(ebd) +
  geom_bar(aes(x = observation_date, color = common_name, fill = common_name)) +
  ggtitle("Observation Date by Species") +
  xlab("Checklist date") + ylab("Count") +
  labs(color = "Species",
       fill = "Species")
```

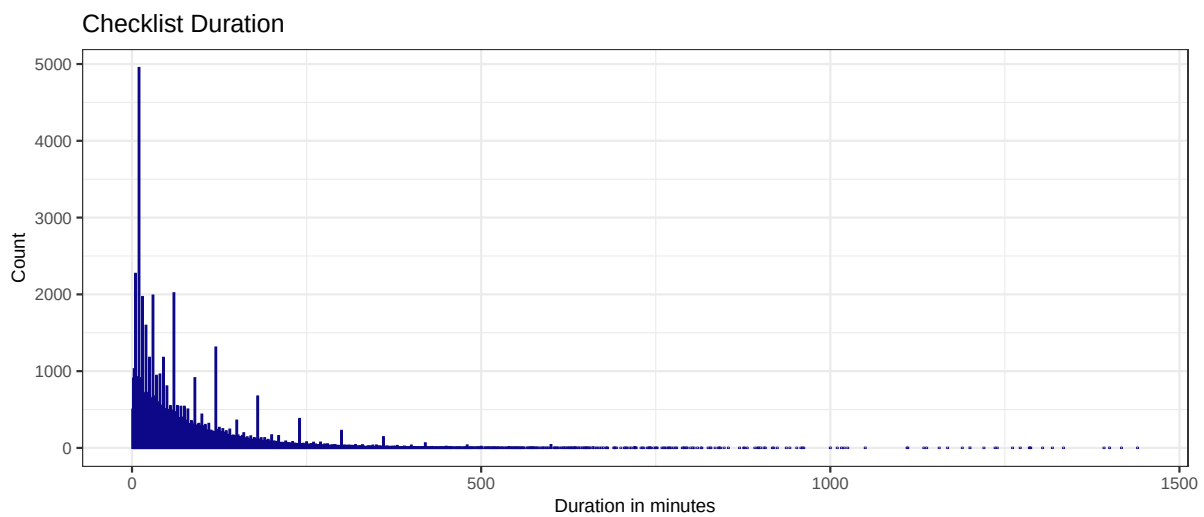


Which types of observation types or counting protocols give more data?

```
ggplot(effort) + # Note the dataset used is not ebd
  geom_bar(aes(x = observation_type)) +
  ggtitle("Observation Type") +
  coord_flip() +
  xlab("Type") + ylab("Count")
```



```
ggplot(effort) + # You can safely ignore the plotting error here
  geom_bar(aes(x = duration_minutes)) +
  ggtitle("Checklist Duration") +
  xlab("Duration in minutes") + ylab("Count")
```



1.3. Pipes- the building block of the 'auk' workflow

'auk' is meant to be a customizable tool to generate your own datasets from eBird raw data. It is designed to work nicely with pipes to create a sequence of commands for how we want the data to be filtered.

Pipes come in two flavors: base (`|>`) and magrittr (tidyverse) (`%>%`). We will use base pipes today in this workshop but know you could use either when creating your own workflows.

Lets take a look at what pipes are meant to do. This is a snippet to filter data within the effort file. We create three objects and rewrite one of them once.

```
# Example: no pipes (stepwise)
tmp1 <- subset(effort, !is.na(duration_minutes))
tmp1 <- transform(tmp1, duration_hours = duration_minutes / 60)
duration_hours <- tmp1$duration_hours
duration_max_no_pipe <- max(duration_hours)

duration_max_no_pipe
```

```
[1] 24
```

With the pipes sequence, we create 1 object and can read the procedure in one sequence. It makes the code more readable.

```
# Example: with base R pipe
duration_max_pipe <- effort |>
  subset(!is.na(duration_minutes)) |>
  transform(duration_hours = duration_minutes / 60) |>
  _$duration_hours |> # "_" passes object to next line of code
  max()

duration_max_pipe
```

```
[1] 24
```

End of section: **Clear your workspace for the next session**

Section 2. Setup and execute filters for eBird raw data in 'auk'

For this portion of the workshop we will use only the ebd file with species observations, in order to learn how 'auk' filters work.

```
library(tidyverse)
library(auk)
source("custom-functions.R")
```

2.1. Set path

Remember, the first step is to setup the path for ‘auk’ to find the eBird files. We will continue working with the small Central American dataset.

```
in_ebd <- "data/ebd_filtered_cam_AOS_2026.txt"
```

2.2. Define ‘auk’ filters

Defining ‘auk’ filters has been compared to conducting your sampling design with already collected data. **This** is where you customize the information you need from a large dataset such as the complete eBird dataset for a country.

The package provides a prefix ‘auk_’ to all functions that conduct a filtering task.

If you are working with large datasets, the filtering step uses `auk_ebd` first, to reference the file of interest instead of reading it into your R session’s memory. By building your entire filter pipeline you are effectively using AWK to help filter large files before having to load them in with `read_ebd`.

```
?auk_ebd()

# Try others such as ?auk_species, ?auk_date to see what they do
```

Lets build a simple set of filters. Remember that to build the sequence we will use the pipe “|>”

The output is a summary of your filters, in this case it indicates the species of interest as the only filter and the rest maintain “all” data.

```
filters1 <-
  # Set EBD file path
  auk_ebd(in_ebd) |>
  # Species: common and scientific names can be mixed
  auk_species(species = "Crested Guan")

filters1
```

Input

EBD: /Users/amv48/Documents/Work & Career/Postdoc /Conferences & Presentations/2026_AOS/eBird workshop/aos-2026_auk-workshop-materials-instructors-copy/data/ebd_filtered_cam_AOS_2026.txt

Output

Filters not executed

Filters

Species: Penelope purpurascens
Countries: all
States: all
Counties: all
BCRs: all
Bounding box: full extent
Years: all
Date: all
Start time: all
Last edited date: all
Protocol: all
Project code: all
Duration: all
Distance travelled: all
Records with breeding codes only: no
Exotic Codes: all
Complete checklists only: no

As filters get more complicated (i.e. more than one species), it may be useful to create objects for the filters.

```
# Define filters outside of pipeline; useful for complicated filters
species <- c("Crested Guan", "Northern Emerald-Toucanet")
species_names <- ebird_species(species, type = "common")
species_names
```

```
[1] "Crested Guan"          "Northern Emerald-Toucanet"
```

```
filters2 <-
  # Set EBD file path
  auk_ebd(in_ebd) |>
  # Species: common and scientific names can be mixed
  auk_species(species = species_names)

filters2
```

Input

EBD: /Users/amv48/Documents/Work & Career/Postdoc /Conferences & Presentations/2026_AOS/eB
workshop/aos-2026_auk-workshop-materials-instructors-copy/data/ebd_filtered_cam_AOS_2026.txt

Output

Filters not executed

Filters

Species: Aulacorhynchus prasinus, Penelope purpurascens
Countries: all
States: all
Counties: all
BCRs: all
Bounding box: full extent
Years: all
Date: all
Start time: all
Last edited date: all
Protocol: all
Project code: all
Duration: all
Distance travelled: all
Records with breeding codes only: no
Exotic Codes: all
Complete checklists only: no

Lets build more filters.

```
## Exploring tailored filters: Thinking about sampling design and reducing variation
filters3 <- auk_ebd(in_ebd) |>
  auk_species(species = species_names) |>
  auk_country("Honduras") |>
  # Date: use standard ISO date format `YYYY-MM-DD`
  auk_date(date = c("*-01-01", "*-12-31"))

filters3
```

Input

EBD: /Users/amv48/Documents/Work & Career/Postdoc /Conferences & Presentations/2026_AOS/eB
workshop/aos-2026_auk-workshop-materials-instructors-copy/data/ebd_filtered_cam_AOS_2026.txt

Output

Filters not executed

Filters

```
Species: Aulacorhynchus prasinus, Penelope purpurascens
Countries: HN
States: all
Counties: all
BCRs: all
Bounding box: full extent
Years: all
Date: *-01-01 - *-12-31
Start time: all
Last edited date: all
Protocol: all
Project code: all
Duration: all
Distance travelled: all
Records with breeding codes only: no
Exotic Codes: all
Complete checklists only: no
```

Exercise # Design your own filters and print them out. The collection of filters can be thought of as the ‘sampling design’ of this eBird data as it describes time, place and other factors tailored to a question.

```
# filters4 <-
```

2.3. Apply filters and save new dataset

None of the filters above were saved in a file that can be used in analyses. The final step is to apply `auk_filter`.

Below we filter, save the data and load it with `read_ebd`

```
# Determine the path and name of your output file
out_ebd <- "data/ebd_filtered_cam_presence.txt"

# Apply the filter
presence_out <- auk_filter(filters3,
                           file = "data/ebd_filtered_cam_presence.txt",
                           overwrite = TRUE) |>

# Read filtered data into R environment
read_ebd()
```

Below we confirm that this dataset only has the species and country data we specified.

```
unique(presence_out$common_name)
```

```
[1] "Northern Emerald-Toucanet" "Crested Guan"
```

```
unique(presence_out$country)
```

```
[1] "Honduras"
```

We can also check how much we have reduced an original eBird file after filtering.

```
# Compare file sizes before and after filtering
get.filesize(in_ebd)
```

```
[1] "17.44 MB"
```

```
get.filesize(out_ebd)
```

```
[1] "247.25 KB"
```

End of section: **Clear your workspace**

Section 3. Zerofilling eBird raw data to obtain a “presence-absence” dataset

Presence-only data are useful for exploring how frequently a species has been detected and where. However, for some analyses, they are not enough to understand what drives species occurrence when there is no information on where the species is absent. This is where **zerofilling** comes in, an ‘auk’ function that can take the eBird data file (ebd) and the sampling file to use the checklist_id information and fill in when a species was or not observed.

```
library(tidyverse)
library(auk)
library(lubridate)
library(cowplot)
library(sf)
source("custom-functions.R")

theme_set(custom.theme())
```

3.1. Set the path for the files

For this zerofilling exercise we will use a different and larger dataset. In honor of the AOS 2026 location, we will use eBird raw data for the three banner species: American Woodcock, Prairie Warbler and Black-capped Chickadee. Observations were filtered for the eastern U.S.

```
in_ebd <- "data/ebd_filtered_us_AOS_2026.txt"
in_effort <- "data/effort_filtered_us_AOS_2026.txt"

## Observe how large these files are compared to those from the previous section
get.filesize(in_ebd)
```

```
[1] "1.14 GB"
```

```
get.filesize(in_effort)
```

```
[1] "1.5 GB"
```

3.2. Applying filters

After identifying the file paths, let's set up some filters. Note that when calling `auk_ebd`, we need to include the path for the sampling file.

And, we will also include a new function to only include **complete** checklists using `auk_complete`. Complete checklists help confirm that we are capturing the most thorough sampling by an observer, including all available and identifiable species.

```
species_names = c("amewoo")

filters <-
  # Set EBD & effort file paths
  auk_ebd(in_ebd, file_sampling = in_effort) |>
  # Species: common and scientific names can be mixed
  auk_species(species = species_names) |>
  # State: two-part 4-6 character codes
  ## (2-letter ISO country code & 1-3 character state code). E.g., "US-NY"
  auk_state(state = "US-MA") |>
  # Date: use standard ISO date format `YYYY-MM-DD`
  auk_date(date = c("2025-01-01", "2025-12-31")) |>
  # Time: 24h format with beginning and end times
  auk_time(start_time = c("17:00", "23:00")) |>
```

```
# Complete: all species seen or heard are recorded (important!)
auk_complete()

filters
```

Input

EBD: /Users/amv48/Documents/Work & Career/Postdoc /Conferences & Presentations/2026_AOS/eBird workshop/aos-2026_auk-workshop-materials-instructors-copy/data/ebd_filtered_us_AOS_2026.txt
 Sampling events: /Users/amv48/Documents/Work & Career/Postdoc /Conferences & Presentations workshop/aos-2026_auk-workshop-materials-instructors-copy/data/effort_filtered_us_AOS_2026.txt

Output

Filters not executed

Filters

Species: Scolopax minor
 Countries: all
 States: US-MA
 Counties: all
 BCRs: all
 Bounding box: full extent
 Years: all
 Date: 2025-01-01 - 2025-12-31
 Start time: 17:00-23:00
 Last edited date: all
 Protocol: all
 Project code: all
 Duration: all
 Distance travelled: all
 Records with breeding codes only: no
 Exotic Codes: all
 Complete checklists only: yes

We filter the eBird and sampling data, save it and upload these new (filtered) data files.

```
f_ebd <- "data/ebd_filtered_amewoo_presabs.txt"
f_effort <- "data/effort_filtered_amewoo_presabs.txt"

## Call AWK to execute filters
f_ebd_effort <- auk_filter(filters,
                           file = f_ebd,
```

```

file_sampling = f_effort,
overwrite = TRUE) |>

read_ebd()

glimpse(f_ebd_effort)

```

```

Rows: 1,032
Columns: 51
$ checklist_id          <chr> "G11916751", "G14123401", "G14127323", "G141~
$ global_unique_identifier <chr> "URN:CornellLabOfOrnithology:EBIRD:OBS300625~
$ last_edited_date      <chr> "2025-08-07 21:09:18.508125", "2025-08-
08 22~
$ taxonomic_order       <dbl> 6064, 6064, 6064, 6064, 6064, 6064, 6064, 60~
$ category              <chr> "species", "species", "species", "species", ~
$ taxon_concept_id      <chr> "avibase-F4829920", "avibase-F4829920", "avi~
$ common_name           <chr> "American Woodcock", "American Woodcock", "A~
$ scientific_name        <chr> "Scolopax minor", "Scolopax minor", "Scolopa~
$ exotic_code           <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ observation_count     <chr> "6", "2", "1", "4", "2", "8", "4", "4", "1",~
$ breeding_code         <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ breeding_category     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ behavior_code         <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ age_sex               <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ country               <chr> "United States", "United States", "United St~
$ country_code          <chr> "US", "US", "US", "US", "US", "US", "US", "U~
$ state                <chr> "Massachusetts", "Massachusetts", "Massachus~
$ state_code           <chr> "US-MA", "US-MA", "US-MA", "US-MA", "US-
MA",~
$ county               <chr> "Middlesex", "Barnstable", "Barnstable", "Mi~
$ county_code          <chr> "US-MA-017", "US-MA-001", "US-MA-001", "US-
M~
$ iba_code             <chr> NA, NA, "US-MA_645", NA, "BIRDLIFE_31394|US-
~
$ bcr_code             <int> 30, 30, 30, 30, 30, 30, 30, NA, 30, 30, 30, ~
$ usfws_code           <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ atlas_block          <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ locality             <chr> "Rock Meadow Conservation Area, Belmont", "B~
$ locality_id          <chr> "L593404", "L4344341", "L1166982", "L593404"~
$ locality_type         <chr> "H", "P", "H", "H", "H", "H", "H", "P", "H",~
$ latitude             <dbl> 42.39980, 41.77671, 41.71652, 42.39980, 42.7~
$ longitude            <dbl> -71.19568, -70.03582, -70.36714, -71.19568, ~

```

```

$ observation_date      <date> 2025-03-16, 2025-02-24, 2025-02-26, 2025-
03~
$ time_observations_started <chr> "18:26:00", "17:44:00", "17:44:00", "17:44:0~
$ observer_id           <chr> "obsr741153", "obsr38456,obsr411957", "obsr1~
$ observer_orcid_id     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ sampling_event_identifier <chr> "S221658457", "S215294673,S215495221", "S215~
$ observation_type      <chr> "Traveling", "Traveling", "Traveling", "Trav~
$ protocol_name         <chr> "Traveling", "Traveling", "Traveling", "Trav~
$ protocol_code         <chr> "P22", "P22", "P22", "P22", "P21", "P22", "P~
$ project_names         <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ project_identifiers   <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ duration_minutes      <int> 89, 20, 25, 45, 13, 45, 20, 55, 30, 80, 45, ~
$ effort_distance_km    <dbl> 1.870, 0.136, 3.749, 0.315, NA, 0.322, NA, 0~
$ effort_area_ha        <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ number_observers      <int> 15, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 6, 2~
$ all_species_reported  <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TR~
$ group_identifier      <chr> "G11916751", "G14123401", "G14127323", "G141~
$ has_media             <lgl> TRUE, FALSE, FALSE, FALSE, TRUE, FALSE, FALS~
$ approved              <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TR~
$ reviewed              <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
$ reason                <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ checklist_comments    <chr> "Brookline Bird Club Family Woodcock Walk- L~
$ species_comments      <chr> NA, "Started penting at 5:46", NA, "First pe~

```

3.3. Zerofill to generate the presence-absence dataset

Execute the zerofill function `auk_zerofill`.

```

presabs_zf <-auk_zerofill(f_ebd,
  sampling_events = f_effort,
  collapse = TRUE)

glimpse(presabs_zf)

```

Rows: 22,242

Columns: 41

```

$ checklist_id          <chr> "S261237820", "S234320228", "S264222830", "S~
$ last_edited_date      <chr> "2025-08-04 15:24:29.828035", "2025-10-
20 18~
$ country               <chr> "United States", "United States", "United St~
$ country_code          <chr> "US", "US", "US", "US", "US", "US", "US", "U~

```

\$ state	<chr> "Massachusetts", "Massachusetts", "Massachus~
\$ state_code	<chr> "US-MA", "US-MA", "US-MA", "US-MA", "US-
MA",~	
\$ county	<chr> "Worcester", "Franklin", "Franklin", "Essex"~
\$ county_code	<chr> "US-MA-027", "US-MA-011", "US-MA-011", "US-
M~	
\$ iba_code	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ bcr_code	<int> 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, ~
\$ usfws_code	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ atlas_block	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ locality	<chr> "Trout Brook Conservation Area", "Deerfield ~
\$ locality_id	<chr> "L1003250", "L10012702", "L10012702", "L1002~
\$ locality_type	<chr> "H", "P", "P", "P", "P", "P", "P", "P", "P", ~
\$ latitude	<dbl> 42.39526, 42.55505, 42.55505, 42.78906, 42.7~
\$ longitude	<dbl> -71.84269, -72.60502, -72.60502, -71.11950, ~
\$ observation_date	<date> 2025-06-29, 2025-05-06, 2025-07-31, 2025-
03~	
\$ time_observations_started	<chr> "20:51:00", "18:00:00", "19:40:00", "17:04:0~
\$ observer_id	<chr> "obsr6803198", "obsr197433", "obsr197433", "~
\$ observer_orcid_id	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ sampling_event_identifier	<chr> "S261237820", "S234320228", "S264222830", "S~
\$ observation_type	<chr> "Traveling", "Traveling", "Traveling", "Trav~
\$ protocol_name	<chr> "Traveling", "Traveling", "Traveling", "Trav~
\$ protocol_code	<chr> "P22", "P22", "P22", "P22", "P22", "P22", "P~
\$ project_names	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ project_identifiers	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ duration_minutes	<int> 42, 50, 65, 57, 90, 69, 46, 60, 57, 65, 80, ~
\$ effort_distance_km	<dbl> 22.273, 1.207, 2.414, 2.124, 3.315, 2.543, 1~
\$ effort_area_ha	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ number_observers	<int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
\$ all_species_reported	<lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TR~
\$ group_identifier	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ checklist_comments	<chr> NA, NA, NA, NA, NA, NA, NA, NA, "1 muskrat",~
\$ scientific_name	<chr> "Scolopax minor", "Scolopax minor", "Scolopa~
\$ breeding_code	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ breeding_category	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ behavior_code	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ age_sex	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
\$ observation_count	<chr> "1", "0", "0", "0", "0", "0", "0", "0", "0", ~
\$ species_observed	<lgl> TRUE, FALSE, FALSE, FALSE, FALSE, FALSE, FAL~

The columns observation_count includes counts for each sampling event.

```
glimpse(presabs_zf$observation_count) # observation
```

```
chr [1:22242] "1" "0" "0" "0" "0" "0" "0" "0" "0" "0" "0" "0" "0" "0" ...
```

```
glimpse(presabs_zf$checklist_id) # for each sampling event
```

```
chr [1:22242] "S261237820" "S234320228" "S264222830" "S221189735" ...
```

While this is already the presence-absence dataset there is some processing necessary for the columns to be useful for future analyses. For example take a look at the range of the counts:

```
range(presabs_zf$observation_count)
```

```
[1] "0" "X"
```

“X” is a valid recording character used by observers when they: do not want to count all individuals or are unsure of how many individuals were detected. Because there is no way of guessing if X means 1 or more, we should remove these for future analyses.

There are other small tweaks needed on the data for better comprehension and use for future analysis.

```
zf_eff_transf <- presabs_zf |>
  mutate(
    # Convert count to integer and X to NA (ignore NA warning!)
    ## X indicates species was present, but a number wasn't recorded
    observation_count = as.integer(observation_count),
    # effort_distance_km to 0 for stationary counts
    effort_distance_km = if_else(observation_type == "Stationary",
                                0, effort_distance_km),
    # Convert duration to hours
    effort_hours = duration_minutes / 60,
    # Convert speed km/h
    effort_speed_kmph = effort_distance_km / effort_hours,
    # Split date into year and day of year
    month = month(observation_date),
    day_of_year = yday(observation_date)
  )
```

Along the way in the filtering process we can take a look at the distribution of these variables to learn about how much data we have.

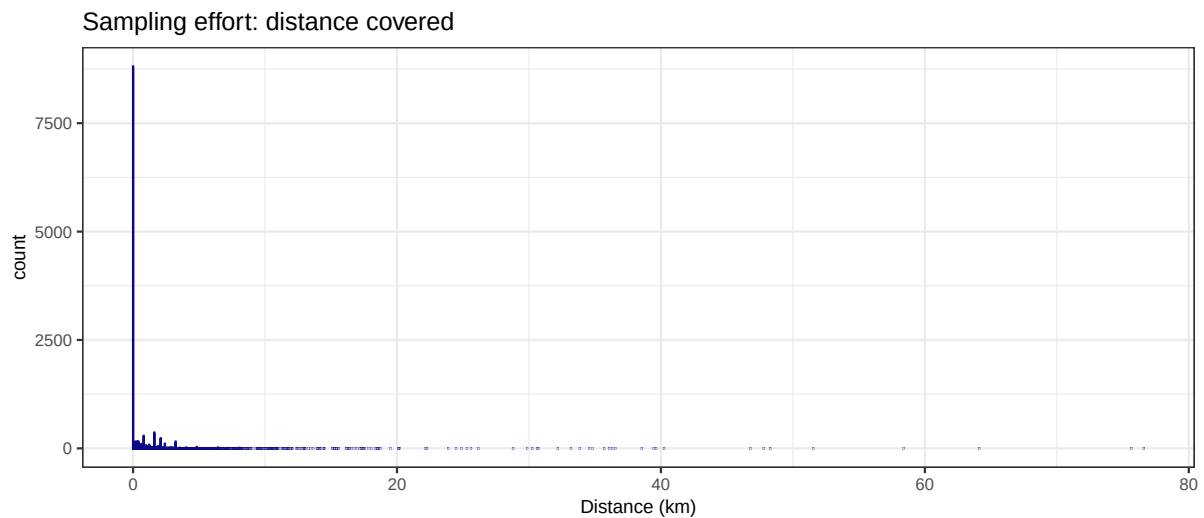
In how many checklists was our species seen?

```
table(zf_eff_transf$species_observed)
```

```
FALSE  TRUE  
21210  1032
```

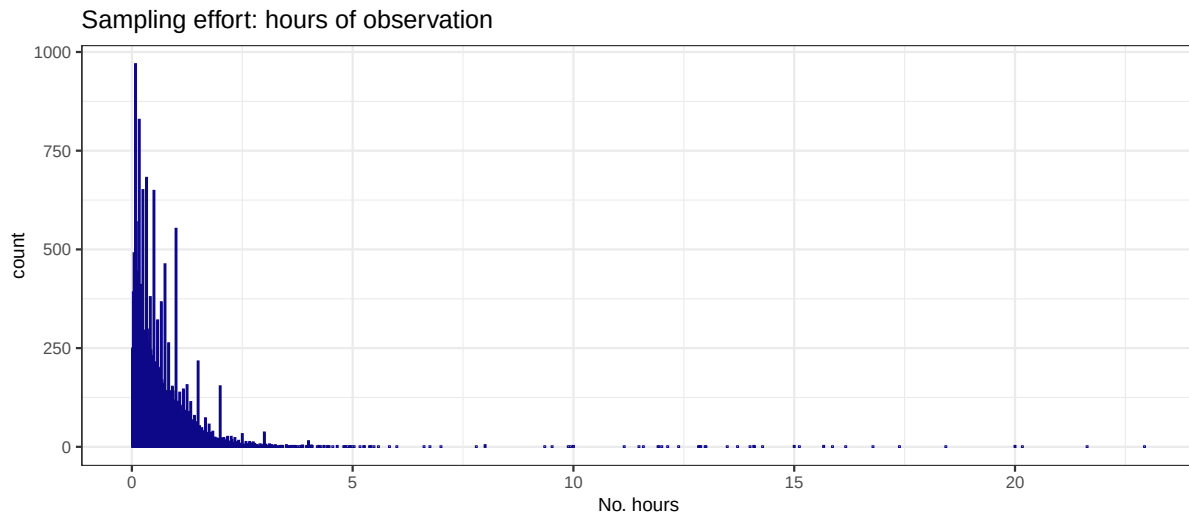
What is the range of the effort in distance covered?

```
ggplot(zf_eff_transf) +  
  geom_bar(aes(x = effort_distance_km)) +  
  ggtitle("Sampling effort: distance covered") +  
  xlab("Distance (km)")
```



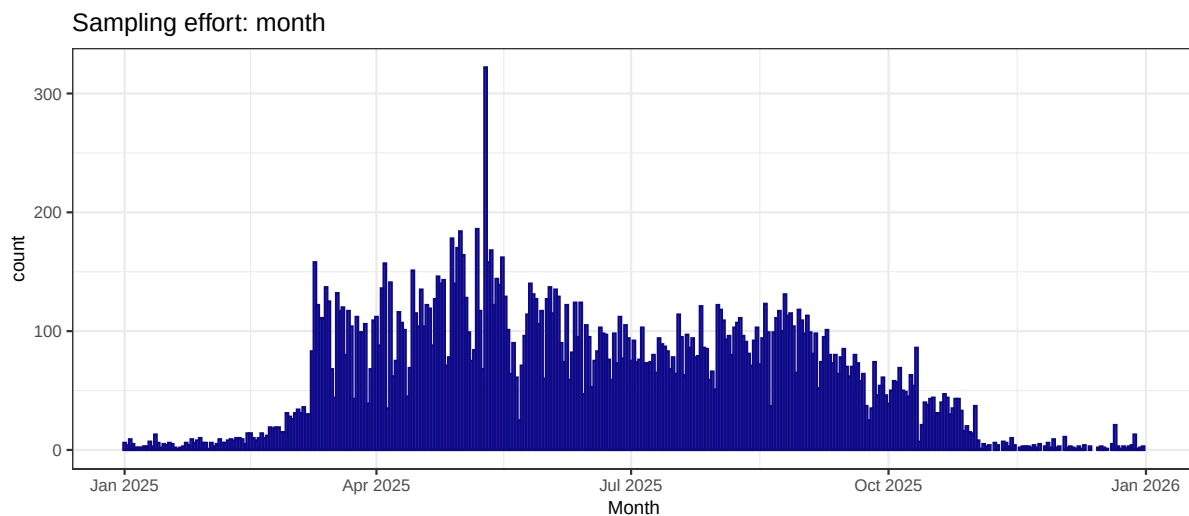
What is the range of the effort of hours of observation?

```
ggplot(zf_eff_transf) +  
  geom_bar(aes(x = effort_hours)) +  
  ggtitle("Sampling effort: hours of observation") +  
  xlab("No. hours")
```



For which time periods is there data?

```
ggplot(zf_eff_transf) +
  geom_bar(aes(x = observation_date)) +
  ggtitle("Sampling effort: month") +
  xlab("Month")
```



These graphs will help determine whether there are time periods, types of protocols (stationary versus walked), or effort levels that should be filtered out because they represent low data amounts.

```
zf_eff_filtered <- zf_eff_transf |>
  filter(observation_type %in% c("Stationary", "Traveling"),
```

```

    !is.na(effort_hours), effort_hours >= 0.17, effort_hours <= 5,
    !is.na(effort_distance_km), effort_distance_km <= 10,
    effort_speed_kmph <= 50,
    number_observers <= 10)

# Compare the filtered and transformed datasets
table(zf_eff_transf$species_observed)

```

```

FALSE  TRUE
21210  1032

```

```

table(zf_eff_filtered$species_observed)

```

```

FALSE  TRUE
15819  815

```

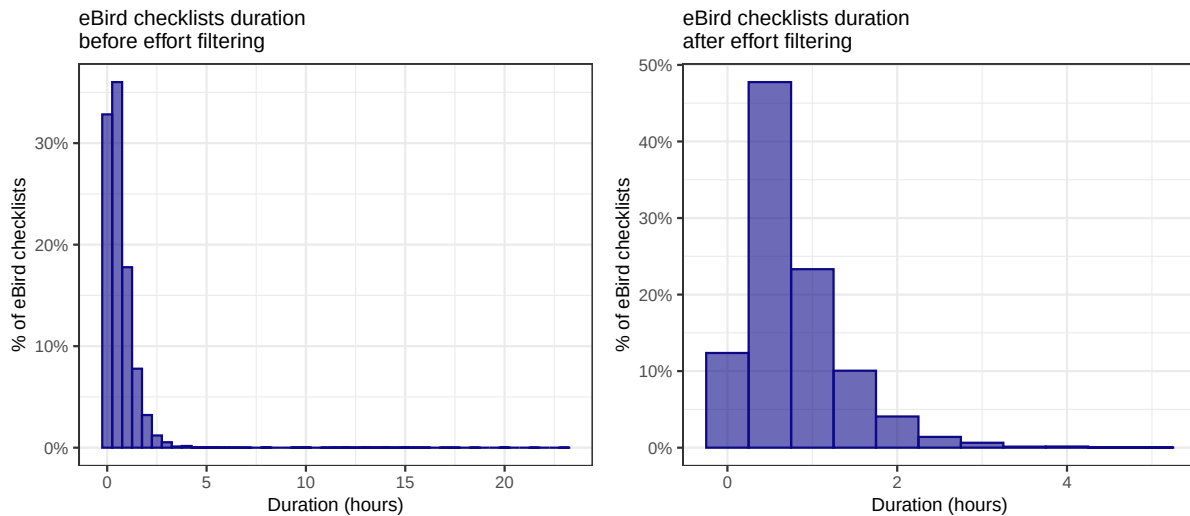
While the prevalence of species did not increase too much, a visual below shows how the data is better constrained to an effort window.

```

plot_grid(
  ggplot(zf_eff_transf) +
    aes(x = effort_hours) +
    geom_histogram(binwidth = 0.5,
                   aes(y = after_stat(count / sum(count)))) +
    scale_y_continuous(limits = c(0, NA), labels = scales::label_percent()) +
    labs(x = "Duration (hours)",
         y = "% of eBird checklists",
         subtitle = "eBird checklists duration\nbefore effort filtering"),

  ggplot(zf_eff_filtered) +
    aes(x = effort_hours) +
    geom_histogram(binwidth = 0.5,
                   aes(y = after_stat(count / sum(count)))) +
    scale_y_continuous(limits = c(0, NA), labels = scales::label_percent()) +
    labs(x = "Duration (hours)",
         y = "% of eBird checklists",
         subtitle = "eBird checklists duration\nafter effort filtering"),
  ncol = 2)

```



This zero-filled data is almost ready for an analysis or further exploration but one last thing to consider after all the desired filters have been applied is to clean the data from columns that are not necessary.

```
zf <- zf_eff_filtered |>
  select(checklist_id,
         latitude,
         longitude,
         observation_date,
         effort_distance_km,
         number_observers,
         scientific_name,
         observation_count,
         species_observed,
         effort_hours,
         month,
         day_of_year)
```

3.4. Final visualization

Because each checklist or sampling event has latitude/longitude data, this data can be presented in maps. It can also be linked to georeferenced environmental covariates for future analyses.

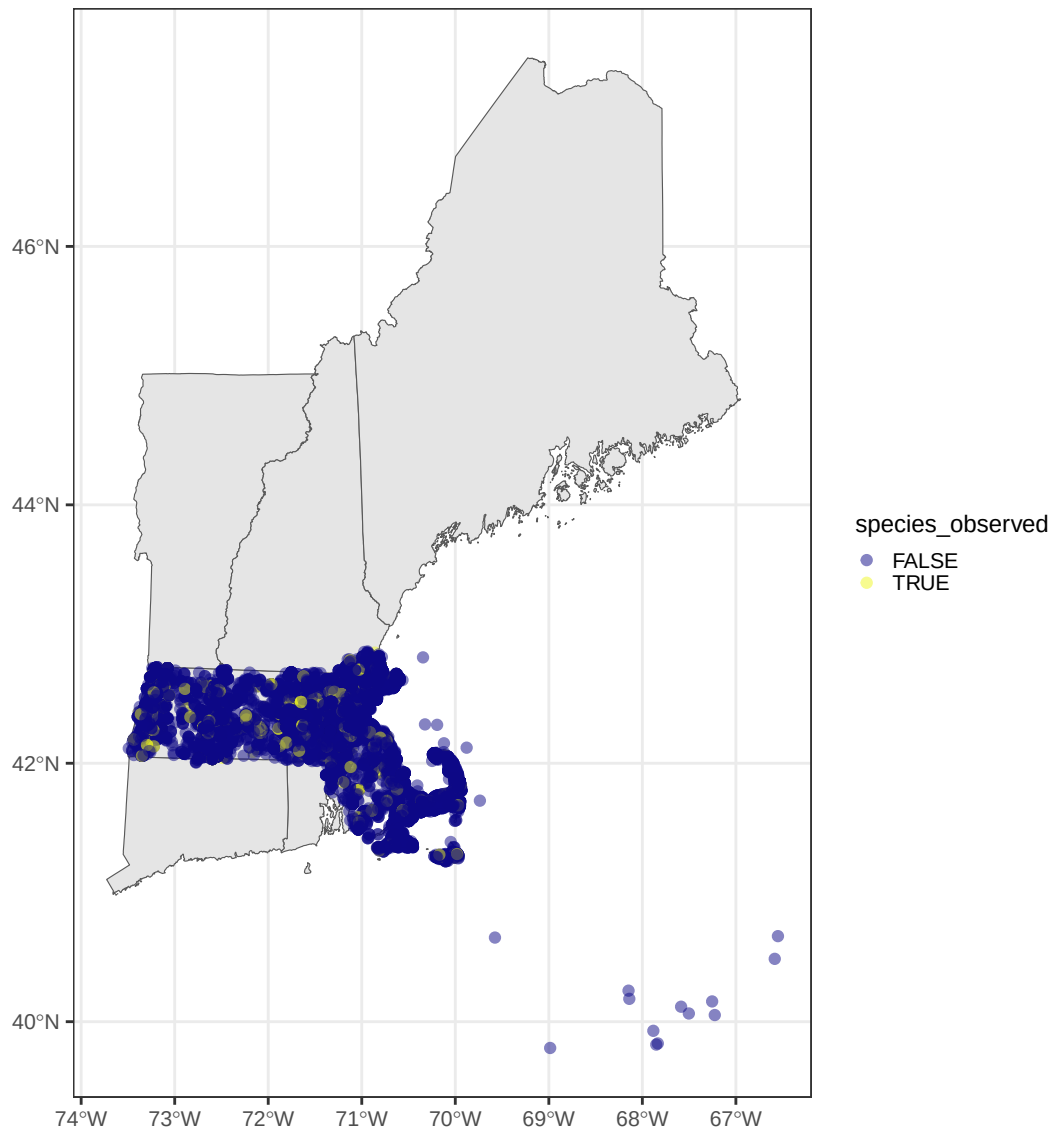
```
# Spatialize zero-filled object
zf_sf <- zf |>
  st_as_sf(coords = c("longitude", "latitude"), crs = 4326)
```

```
# Load GIS and shapefile data
ee_landcover <- read.csv("data/gis/EarthEnv-landcover-northeast.csv") |>
  left_join(landcover_palette) # Add map colors for nice plotting

ne_states_sf <- read_sf("data/gis/usa-northeast-states.shp") |>
  st_transform(crs = 4326)
```

Plot the locations where the species was observed.

```
ggplot() +
  geom_sf(data = ne_states_sf) +
  geom_sf(data = zf_sf, aes(color = species_observed), alpha = 0.5)
```

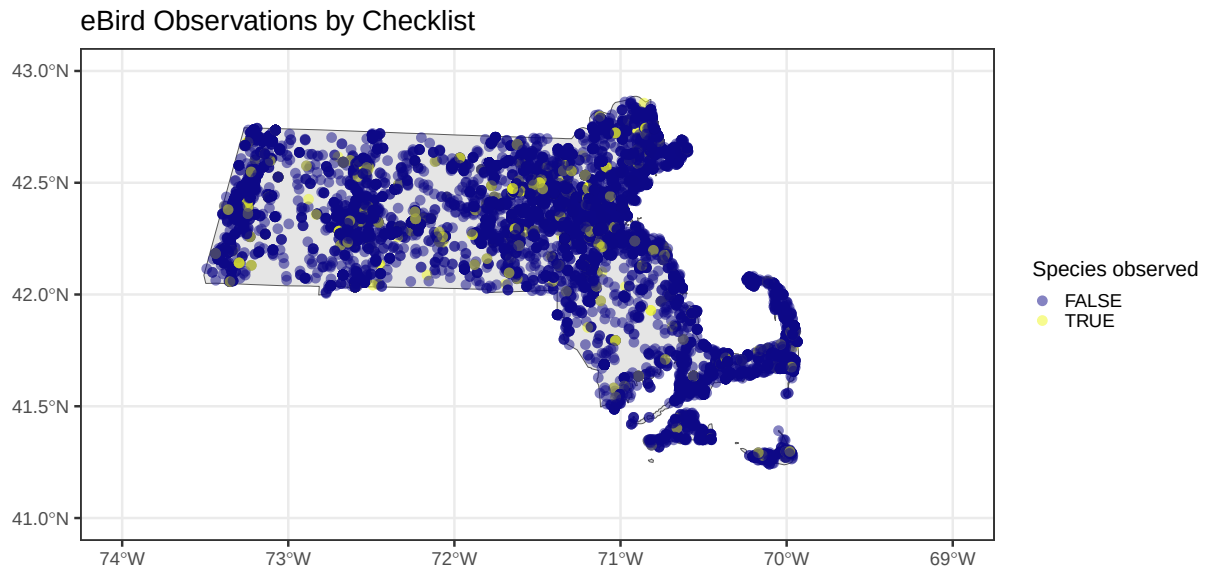


Lets modify the map to visualize the state of interest.

```
state_sf <- ne_states_sf[ne_states_sf$NAME == "Massachusetts", ]
# Clip checklists to land; safe to ignore warning message
zf_clip_sf <- st_intersection(zf_sf, state_sf)

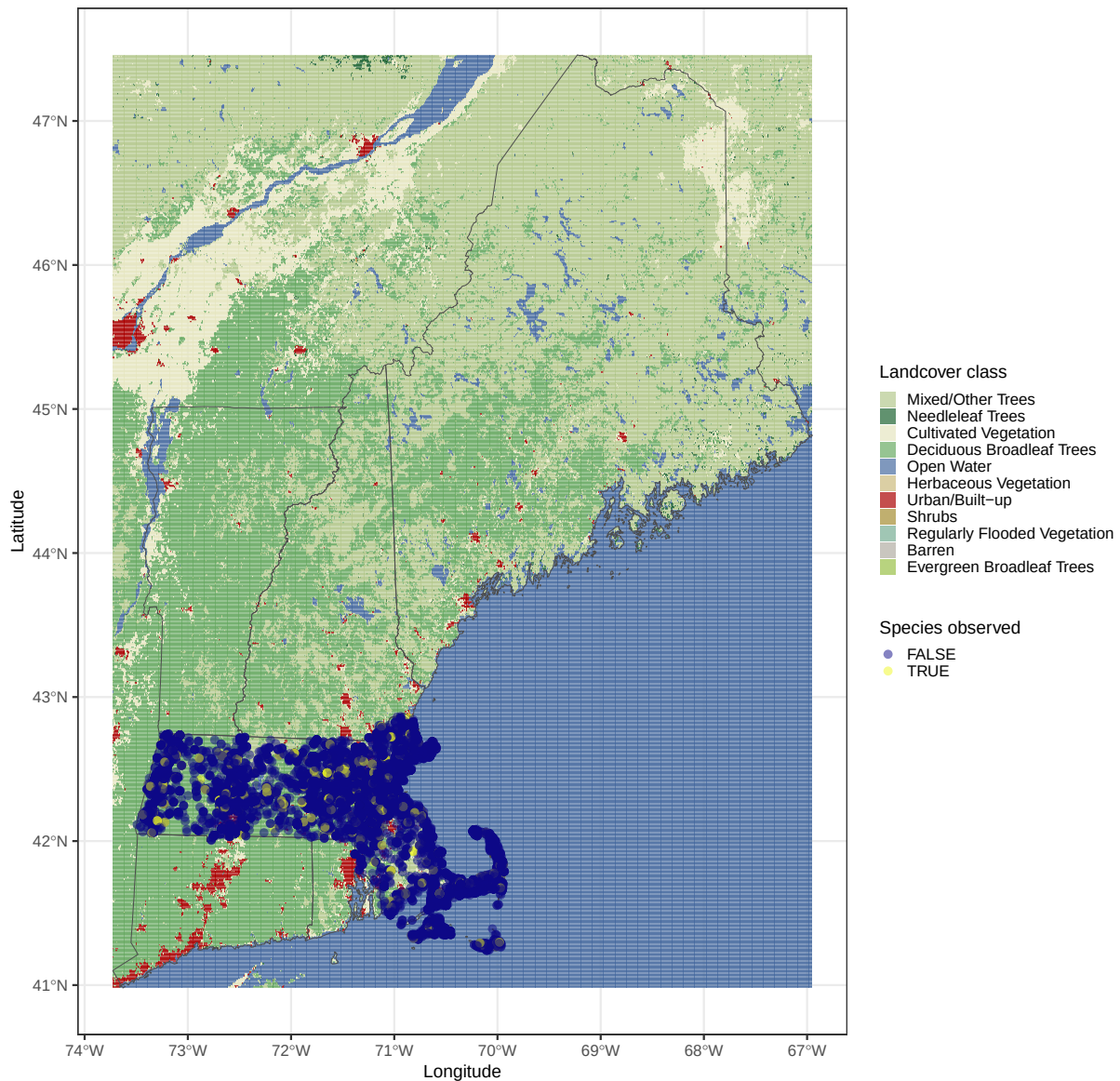
## Clipped map
ggplot() +
  geom_sf(data = state_sf) +
  geom_sf(data = zf_clip_sf, aes(color = species_observed), alpha = 0.5) +
  coord_sf(xlim = c(-74, -69), ylim = c(41, 43)) +
```

```
ggtitle("eBird Observations by Checklist") +
labs(color = "Species observed")
```



With your dataset you can plot it alongside other spatial datasets of interest such as this example with landcover.

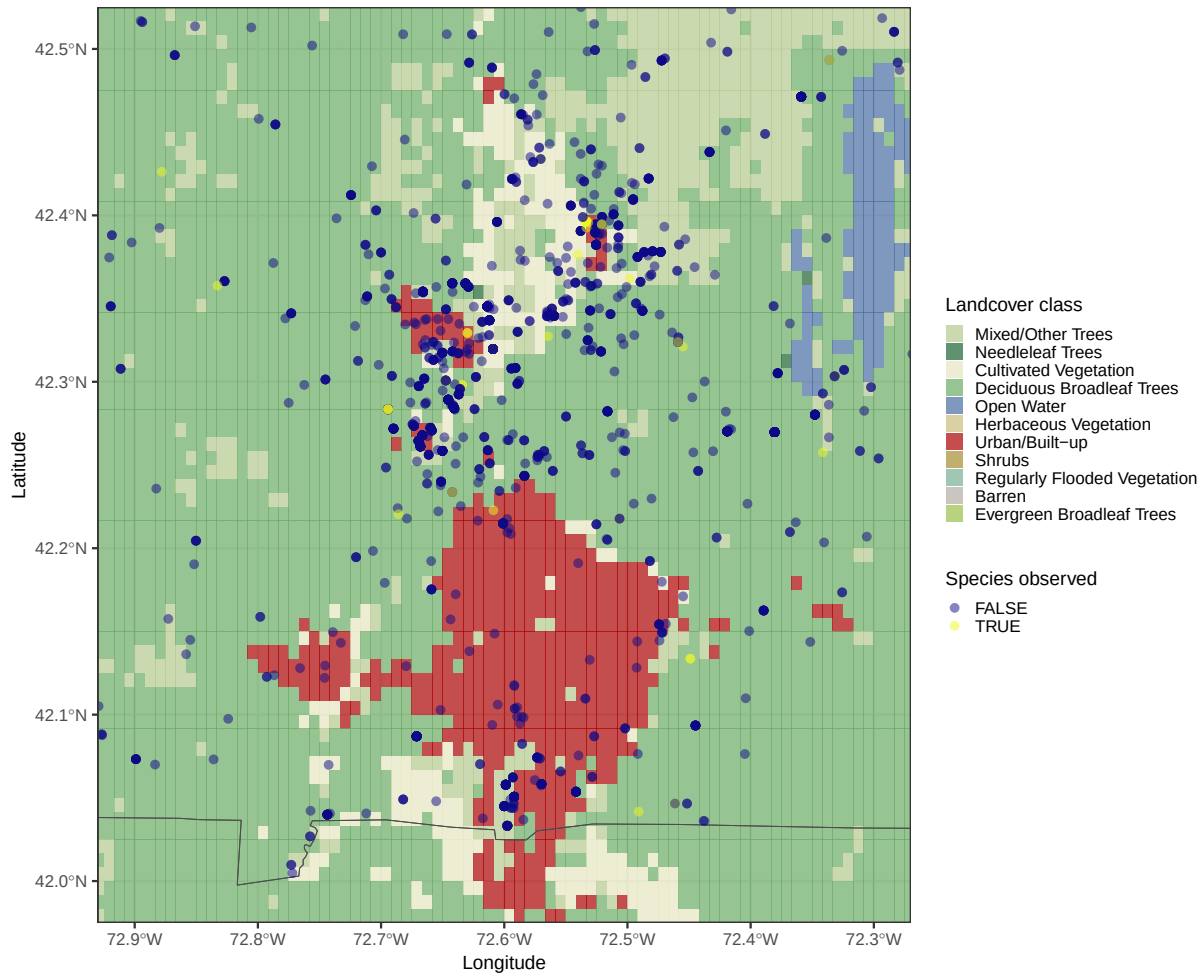
```
ggplot() +
  geom_tile(data = ee_landcover,
            aes(x = x, y = y,
                fill = class_names), alpha = 0.7) +
  geom_sf(data = ne_states_sf, color = "grey30", fill = NA) +
  geom_sf(data = zf_clip_sf, aes(color = species_observed), alpha = 0.5) +
  scale_fill_manual(breaks = unique(ee_landcover$class_names),
                    values = unique(ee_landcover$colors)) +
  xlab("Longitude") + ylab("Latitude") +
  labs(color = "Species observed",
       fill = "Landcover class")
```



Try zooming in (using `coord_sf`) to see if you can identify any associations with observations of your species and landcover types.

```
ggplot() +
  geom_tile(data = ee_landcover,
            aes(x = x, y = y,
                fill = class_names), alpha = 0.7) +
  geom_sf(data = ne_states_sf, color = "grey30", fill = NA) +
  geom_sf(data = zf_clip_sf, aes(color = species_observed), alpha = 0.5) +
  coord_sf(xlim = c(-72.9, -72.3), ylim = c(42, 42.5)) + # Adjust to your liking
```

```
scale_fill_manual(breaks = unique(ee_landcover$class_names),
                  values = unique(ee_landcover$colors)) +
xlab("Longitude") + ylab("Latitude") +
labs(color = "Species observed",
     fill = "Landcover class")
```



End of section and workshop! Explore the applications examples and best of luck in your future analyses with eBird data :)